

Migrating Java Services to Eclipse Vert.x: Measuring the Trade-offs

Sai Praneeth Kalimisetty — Independent Researcher, Phoenix, Arizona, United States

October 3, 2026 — Working draft; empirical validation pending

Abstract

Migrating a blocking Java service to an event-driven implementation changes where requests wait, how failures propagate, and which resources limit throughput. These changes do not guarantee lower latency or cloud costs. This article presents a small database-backed reference implementation and a measurement protocol for evaluating a migration to Eclipse Vert.x. It explains how to preserve workload semantics, separate application concurrency from database capacity, and compare cost at an equivalent service-level objective. The repository currently provides implementation and benchmark scaffolding rather than validated performance results. Its intended contribution is a practical evaluation method that helps engineers test migration assumptions before extending them to production systems.

Three actionable insights

- Measure database capacity and queueing before changing the HTTP execution model; a non-blocking client does not remove downstream bottlenecks.
- Compare successful work at the same offered load, latency target, and error budget; peak request counts alone are insufficient.
- Retain raw measurements and a complete cost model before turning a benchmark difference into a claimed operational saving.

Start with a bottleneck, not a framework

A Java modernization effort often begins with a familiar symptom: response time rises while requests accumulate. The symptom does not identify its cause. A servlet thread pool, a database connection pool, a slow query, or a constrained processor may each limit progress. Changing the programming model without locating that limit can move the queue instead of improving the service.

Eclipse Vert.x supports event-driven request handling and asynchronous database access [1, 2]. A handler can initiate a database operation and allow its event loop to process other events while the operation is pending. In a conventional JDBC request path, a worker thread waits for the result. This distinction motivates measurement; it does not establish that the asynchronous implementation is faster for every workload.

This article describes a reference experiment, not a completed enterprise migration. No production deployment, measured speedup, or verified cost saving is claimed. The existing repository has no archived raw measurements establishing the quantitative results in an earlier manuscript. Those values have therefore been excluded from this draft and retained separately for audit.

A deliberately small reference system

The reference repository contains a Spring Boot 3.4.5 service and a Vert.x 4.5.13 service, configured for Java 21. Both query a PostgreSQL fixture by item identifier and include a requested delay through PostgreSQL's `pg_sleep`. Each returns a small success marker for a valid fixture row. The services use different HTTP stacks and database clients: Spring MVC with JDBC/HikariCP versus Vert.x Web with the asynchronous PostgreSQL client. Spring Boot's documented JDBC configuration provides the background for the baseline [3].

The default database pool size is 20 for each service. The servlet configuration limits its worker pool to 200 threads. The reactive application deploys one verticle; it does not implement the event-bus decomposition described in the earlier manuscript. It should not be described as a distributed microservice system or a full enterprise monolith migration.

Only one service should receive measured load at a time. The two implementations use the same fixture operation for valid inputs, but their error handling is not equivalent: malformed parameters and absent rows can produce different behavior. The present workload uses valid parameters and an existing row. A production migration requires explicit equivalence tests for invalid input, missing records, dependency failure, transaction boundaries, and cancellation before performance comparisons can support a deployment decision.

The synthetic database wait isolates one kind of blocking, not a realistic business workload. It does not model complex joins, contention among transactions, external service calls, or serialization-heavy processing. It is useful for examining queue formation, while realistic conclusions require additional representative operations.

Understand where waiting work lives

Little's Law relates the average number of requests in a defined system, its long-run throughput, and mean time in that system: $L = \lambda W$ [4]. Here W includes both queueing and service time. Measurements must use the same boundary and a stable interval; a growing overload queue is not a steady-state experiment.

The number of requests in a service is not automatically its platform-thread count. A bounded servlet pool can leave requests waiting outside its worker threads. Conversely, an asynchronous service retains request state, buffers, callbacks, and database-pool waiters. A small event-loop pool does not imply constant process memory. JVM heap, committed thread stacks, native allocations, direct buffers, and connection state all need consideration.

A simple capacity calculation helps identify misleading experiments. If each of 20 database connections executes one query at a time and every query occupies that connection for at least 50 milliseconds, the idealized ceiling is $20 / 0.05 = 400$ queries per second, before additional overhead. This is an illustrative bound, not an observed result; pipelining and actual connection behavior must be checked. It explains why increasing HTTP concurrency can increase waiting without increasing completed work.

Total JVM thread count must also be measured rather than inferred from the configured event-loop count. Worker threads, garbage collectors, database clients, and other runtime services contribute. Avoid attributing a throughput difference to context switching unless profiling or a controlled intervention supports that explanation.

Design comparisons that answer a decision

Three questions guide the proposed evaluation. Under what workload conditions does either implementation meet the required latency and error target? Which resource saturates first? How much provisioned capacity does each implementation need to meet the same target?

Begin with equivalent request semantics and a fixed environment. Record the exact repository revision, JDK build, JVM arguments, framework and driver versions, database configuration, connection limits, and container resource limits. Record the load-generator host separately. Container defaults and moving image tags are inadequate descriptions of a reproducible experiment; retain resolved image digests.

The current k6 ramp is an exploratory closed workload: a virtual user waits for its response and a short pause before issuing another request. As the service slows, its offered request rate can fall. A ramp-wide p99 consequently cannot stand in for p99 at a particular load plateau. Use independent fixed-load measurement windows for comparisons, and add an arrival-rate experiment when the question concerns externally arriving traffic. Record dropped iterations, timeouts, and load-generator saturation as well as completed requests [5].

Warm up the JVM and connection pools before each measured interval. Specify the warm-up rule in advance, preserve its logs, and exclude it from the measurement summary. Alternate or randomize implementation order across repeated runs and retain individual results. Report run-to-run variation, not only the most favorable run. Do not average percentiles and describe that average as the percentile of pooled requests.

For each load point, collect successful requests per second, response-time distributions, HTTP errors, semantic-check failures, CPU utilization, process RSS, heap behavior, garbage-collection activity, threads, and database metrics. Compare overload and recovery, not only the last successful load point. A high request rate with failed requests is not useful throughput.

Separate the changes introduced by migration

This reference comparison changes the HTTP framework and database client together. It therefore evaluates two complete configurations; it cannot isolate reactive execution as the sole cause of a difference. Changes to pool size, query pipelining, timeout policy, or deployment topology add further confounders.

A stronger study would introduce intermediate comparisons with matched pool and resource limits, measure each change, and report the settings that differ. A Java virtual-thread baseline is also relevant to a modernization decision because it offers a different way to support many waiting operations without requiring the same asynchronous application structure [6]. That baseline is a proposed extension and is not implemented or evaluated here.

Include workload conditions under which reactive execution may offer little benefit, such as a database already at capacity or an application dominated by CPU work. Report added migration effort, debugging difficulties, and operational requirements alongside performance. A useful practitioner result explains when to keep the existing architecture as well as when to change it.

Calculate cost at equivalent service quality

Lower memory consumption in one process does not by itself lower a cloud bill. Billing depends on the resources provisioned and the pricing model. Define the minimum deployment that sustains the

chosen arrival rate while meeting the same p99 target and error budget, then calculate its cost.

For each implementation, monthly cost should include application instance count times hourly price times billed hours, database capacity, storage, network charges, load balancing, monitoring, and any CPU-credit charges. State the region, price date, purchase model, and exclusions. Report migration engineering effort separately if the claim concerns total economic benefit rather than infrastructure cost.

The earlier manuscript named AWS t3.medium instances. These are burstable instances, so CPU-credit balance and credit mode can affect sustained performance and charges [7]. Either record those conditions throughout every run or use a non-burstable comparison environment. Identical instance names alone do not guarantee identical effective CPU availability.

Cost savings should be expressed as (baseline cost minus treatment cost) divided by baseline cost, with uncertainty from measured capacity estimates. Until measurements and pricing inputs are retained, cost differences remain hypotheses. This draft makes no dollar or percentage savings claim.

Limits and next evidence

The repository is a teaching and experimentation scaffold. It lacks archived publication-grade runs, confidence intervals, production traces, and a measured migration-effort account. The synthetic fixture is narrow, the services' error paths differ, and the comparison changes multiple software components. These limitations prevent general claims about enterprise modernization.

Before an empirical submission, the evidence package needs a documented environment, repeated raw runs, analysis tied to exact revisions, and a related-work comparison establishing what the findings add to prior evaluations. If the work is submitted as a practitioner method article instead, its contribution and examples still need editorial assessment; removing unsupported results does not establish novelty or acceptance readiness.

Conclusion

Reactive migration should be evaluated as a change in resource use, queue placement, failure handling, and engineering effort. The reference implementation provides a starting point for testing those effects. A defensible decision requires equivalent workloads, explicit limits, evidence from repeated measurements, and a cost comparison at the same service quality. Framework choice is the outcome of that evaluation, not a substitute for it.

Acknowledgment and AI-use disclosure

OpenAI ChatGPT assisted with restructuring and drafting this revision, auditing arithmetic and references, and revising repository benchmark tooling. The author must review the generated material, verify technical claims and source attribution, and confirm the complete disclosure against IEEE's current policy before submission. ChatGPT did not supply measured benchmark results.

References

1. Eclipse Vert.x, "Vert.x Core," official documentation. <https://vertx.io/docs/vertx-core/java/> (accessed October 3, 2026). Documentation may describe newer releases; use the pinned implementation version when reproducing.

2. Eclipse Vert.x, “Reactive PostgreSQL Client,” official documentation. <https://vertx.io/docs/vertx-pg-client/java/> (accessed October 3, 2026).
3. Spring, “SQL Databases,” Spring Boot reference documentation. <https://docs.spring.io/spring-boot/reference/data/sql.html> (accessed October 3, 2026).
4. J. D. C. Little, “A Proof for the Queuing Formula: $L = \lambda W$,” Operations Research, vol. 9, no. 3, pp. 383–387, 1961. <https://doi.org/10.1287/opre.9.3.383>
5. Grafana Labs, “Results output” and “Custom summary,” k6 documentation. <https://grafana.com/docs/k6/latest/get-started/results-output/> and <https://grafana.com/docs/k6/latest/results-output/end-of-test/custom-summary/> (accessed October 3, 2026).
6. OpenJDK, “JEP 444: Virtual Threads.” <https://openjdk.org/jeps/444> (accessed October 3, 2026).
7. Amazon Web Services, “Key concepts for burstable performance instances,” Amazon EC2 User Guide. <https://docs.aws.amazon.com/AWSEC2/latest/UserGuide/burstable-credits-baseline-concepts.html> (accessed October 3, 2026).

Author biography

Sai Praneeth Kalimisetty is an independent researcher based in Phoenix, Arizona, United States. His work in this repository concerns Java service modernization and performance evaluation. Contact: kalimisettypraneeth@gmail.com. Author confirmation and an author-supplied photograph are required before submission.